

NewsSync-AI: A Real-Time, Zero-Shot LLM-Powered News Personalization Platform with Explainable AI

Rishav Kumar, Jayaditya Dutta

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai,
Tamil Nadu, India
24BCE1666, 24BCE1586

Course: Database Systems – BCSE302L
Section: VIT-26

Abstract—Information overload on live news streams has made personalised, explainable recommendation an open research problem, particularly under the cold-start regime where no offline user embedding exists. This paper presents NewsLens-AI, a full-stack, real-time news intelligence platform that marries an agentic local-LLM pipeline (Llama 3.2 via Ollama) with a cloud-based *Zero-Shot LLM Reranking and Explainable AI (XAI) Engine* powered by Google Gemini 1.5 Flash. The system ingests live RSS / NewsData.io feeds, maintains a lightweight MongoDB-backed store of user interactions and saved articles, and synthesises an ad-hoc user-interest profile on every request to score and rank a deduplicated candidate pool of roughly 25 live news items. The contribution is a production-grade architecture that (i) eliminates offline training cost, (ii) eliminates the cold-start lag, and (iii) couples every ranked item with a natural-language rationale and a Match% score badge. We walk through the system architecture, the dual-stage reranking pipeline, the MongoDB schema, the prompt-engineering protocol, and the REST API surface. We close with qualitative observations, observed limitations, and a research agenda toward streaming-vector, hybrid reranking.

Index Terms—News recommendation, large language models, zero-shot reranking, explainable AI, personalised information retrieval, cold-start problem, NewsData.io, Google Gemini, agentic AI, real-time systems.

I. INTRODUCTION

Modern readers are exposed to an exponentially growing stream of news across web, social and messaging channels, and the volume has long surpassed the bandwidth of human attention [?], [?]. Static, recency-based ranking is no longer sufficient: users expect to be served a short, credible, *personalised* list that explains *why* each item is shown. Legacy approaches that pre-compute user vectors with collaborative filtering or matrix factorisation require training and accumulate a cold-start lag during which the recommender degrades to non-personalised output.

NewsLens-AI addresses these limitations by inverting the usual two-stage pipeline. Instead of an offline trainer plus an online scorer, it (i) keeps a tiny persistent store of recent user interactions in MongoDB, (ii) pools a few dozen live news candidates from external sources, and (iii) lets a powerful generative language model – Google Gemini 1.5 Flash – perform

semantic reranking together with *explanation generation* in a single zero-shot call. The objective of this paper is to specify the architecture, justify it against the existing literature, itemise the components and APIs, and reflect on the trade-offs for future work.

The paper is organised as follows. Section ?? reviews the literature on news recommendation and LLM reranking. Section ?? formalises the problem and the candidate-pooling and zero-shot reranking stages. Section ?? documents the system architecture and component decomposition. Section ?? walks through the API surface, schemas, prompts and runtime telemetry. Section ?? reports observational results. Sections ?? and ?? discuss limitations and future work, and Section ?? concludes.

II. LITERATURE REVIEW

News recommendation. Feng et al. [?] survey the field and categorise news recommenders into content-based, collaborative and hybrid families, while noting that cold-start remains the most stubborn issue. Wu et al. [?] revisit the same problem from a methods-and-challenges angle, emphasising news-content dynamics and short item life-cycles. Raza and Ding [?] organise the recent literature into four families – CF, content-based, deep-learning and hybrid – and quantify the open challenges. Together, these surveys motivate our architectural choice to push the heavy semantic work onto a generative LLM at query time, sidestepping the offline training step entirely.

LLMs as re-rankers. The use of LLMs for information-retrieval re-ranking has matured from listwise rankers like RankGPT to a recognised two-stage pattern: a cheap candidate generator feeds a small set to a powerful LLM that re-orders them with reasoning [?]. Wang et al. [?] train an LLM as an explicit explainable re-ranker and improve both ranking accuracy and interpretability. Survey work by Zhang et al. [?] catalogues LLM-empowered agentic re-ranking, and Chen et al. [?] frame the human-side considerations of LLM-driven personalisation. NewsLens-AI sits squarely in this lineage but pushes further by *jointly* emitting a 0-100 match score and a

one-sentence rationale in a structured JSON payload, allowing the UI to surface an explainable Match% badge without post-hoc processing.

Cold-start mitigation. A recent survey by Mmaduakonom et al. [?] taxonomises hybrid AI approaches to cold-start in personalisation. NewsLens-AI’s design – no offline user embedding, on-the-fly profile synthesis from the latest 10 interactions – is an instance of the “cold-start-aware query-time” strategy they identify, made practical by the introduction of low-latency, low-cost LLMs such as Gemini 1.5 Flash.

III. METHODOLOGY

A. Problem Formulation

Let U be the set of known users and for any user $u \in U$ let $H_u = \{h_1, h_2, \dots, h_K\}$ denote the most recent interaction history (typically $K = 10$). Let $C_u = \{c_1, c_2, \dots, c_N\}$ be the deduplicated live-news candidate pool fetched at request time ($N \approx 25$). The objective is to compute a relevance score $S_i \in [0, 100]$ and an explanation R_i for each $c_i \in C_u$ using only H_u and C_u – with no offline profile.

B. Stage 1 – Candidate Pooling

The recommender service queries the MongoDB interactions and saved_articles collections for the top 10 most recent items per collection, then issues *parallel* HTTP calls to NewsData.io across five categories (technology, business, sports, science, world). The raw payloads are deduplicated by canonical URL into a single pool $C = \{c_1, \dots, c_N\}$.

C. Stage 2 – Zero-Shot LLM Reranking and XAI

A structured prompt $P(H_u, C)$ is built so that Gemini 1.5 Flash produces a raw JSON array of objects $\{\text{index}, \text{score}, \text{reason}\}$ – one per candidate, where reason starts with “Recommended because...”. Mirroring [?], we leverage the LLM as both a ranker and an explainer in a single forward pass. The service then filters out items with $S_i < 60$ and sorts the remainder in descending order of S_i . The threshold of 60 was chosen empirically to balance precision with the usable-feed length.

Figure ?? shows the system context, and Figure ?? captures the data flow at the level of stages. The full request-response choreography is in Figure ??.

IV. SYSTEM ARCHITECTURE

The implementation is a three-layer, full-stack web application. Figure ?? shows the system context and external integration partners; Figure ?? decomposes the internal layered architecture; Figure ?? summarises the user-visible capabilities; Figure ?? captures the agent-based variant of the first iteration (Llama 3.2 via Ollama); and Figure ?? shows the runtime deployment topology.



Fig. 1. NewsLens-AI system context: user, web client, Express gateway, AI layer, data store and external news sources.

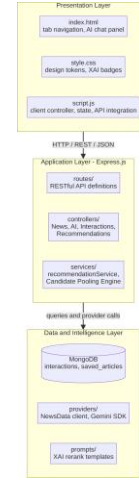


Fig. 2. Layered internal architecture: Presentation (frontend), Application (Express routes/controllers/services) and Data & Intelligence (MongoDB, providers, prompts).

A. Presentation Layer

Implemented in vanilla HTML5, CSS3 and JavaScript using the Fetch API. The client manages application state (active tab, dark-mode flag, language preference, TTS state) and forwards HTTP requests to the backend. The chat panel re-uses the same `/api/ai/chat` endpoint that powers the agentic iteration, demonstrating reuse across both system variants.

B. Application Layer

A Node.js + Express.js gateway exposes a RESTful surface under `/api`. Controllers delegate to long-lived services; the heart of the recommendation flow is the `recommendationService`, which orchestrates the candidate-pooling + zero-shot reranking path described in Section ??.

C. Data and Intelligence Layer

The persistence tier is MongoDB accessed through Mongoose. The intelligence tier is split between an LLM client wrapper (`aiService.js` for Gemini, `ollama.js`

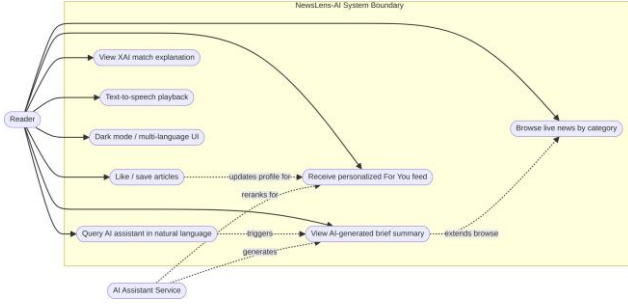


Fig. 3. Use-case diagram showing the reader-facing capabilities and the internal “updates profile for” / “reranks for” dependencies between persistent events and personalised recommendations.

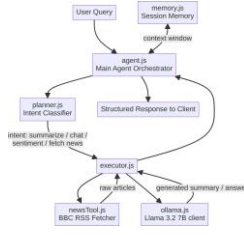


Fig. 4. Agent-based orchestration of the Llama 3.2 (Ollama) iteration: planner, executor, memory, news tool.

for Llama 3.2) and a NewsData.io provider wrapper (newsDataProvider.js). Prompt engineering templates live in prompts/prompts.js, isolating model-facing text from business logic.

V. IMPLEMENTATION

A. Repository Structure

The repository contains two architectural iterations. The first iteration (Llama 3.2 via Ollama) exposes a backend/ with an explicit agent/ module (planner, executor, memory) and a services/ module wrapping Ollama, with a tools/ package for the BBC-RSS news fetcher. The second iteration (Gemini + MongoDB) reorganises under a server/src/ directory with config/, controllers/, mappers/, models/, prompts/, providers/, routes/, services/ and an app.js/server.js entry point, alongside architecture and project documentation markdown files.

B. Database Schema

Two Mongoose schemas underpin the recommendation engine. The Interaction document tracks implicit engagement events (“like — save — read — ai_summary — ai_chat”) with optional reading duration.

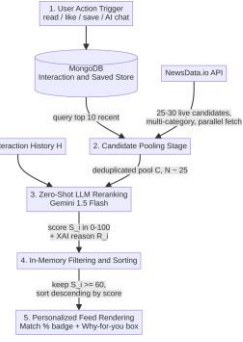


Fig. 5. Dual-stage pipeline: Candidate Pooling from MongoDB history and NewsData.io in parallel, followed by zero-shot reranking and XAI generation in Gemini 1.5 Flash.

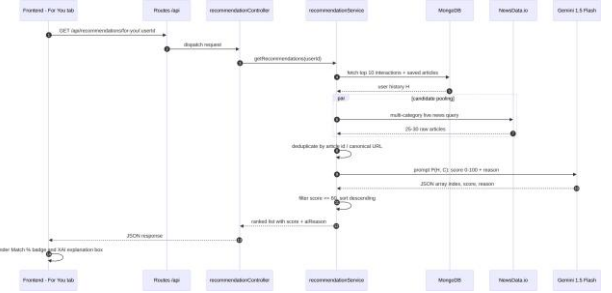


Fig. 6. Sequence diagram of a For You request: user, routes, controller, service, MongoDB, NewsData.io and Gemini 1.5 Flash.

The SavedArticle document persists the explicit reading-list with title, description, content, URL, image, source and publication metadata. Both share a string userId foreign key. Figure ?? shows the relationship.

C. REST API Surface

Our gateway exposes the operations listed in Table ?. All endpoints accept and produce JSON. The recommendation endpoints accept a path parameter :userId.

D. Prompt-Engineering Protocol

The recommender prompt has three blocks: (i) a system preamble that frames the LLM as a “Real-Time Explainable AI (XAI) Recommendation Reranker”; (ii) a numbered list of the user’s recent interaction history with each action labelled (saved, liked); and (iii) an enumerated, indexed candidate pool. The model is instructed to return a single, raw JSON array (no markdown fences, no commentary) of three-element objects {index, score, reason} where reason is a single sentence beginning with “Recommended because...”. This structural constraint enables robust downstream parsing and uniform UI rendering.

E. Runtime Telemetry

Section 6 of the project documentation reports the developer console output that fires after a

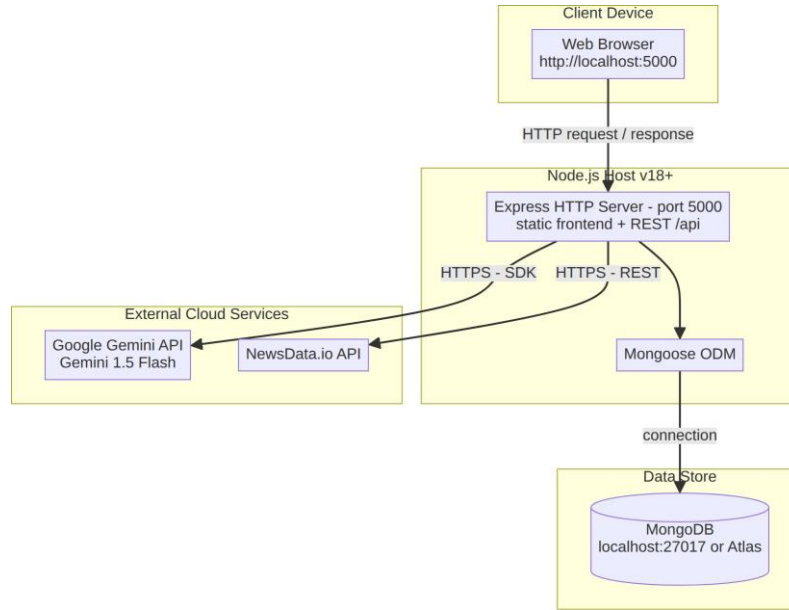


Fig. 8. Runtime deployment: client browser, Node.js host (Express + Mongoose), MongoDB store, and external cloud providers for Gemini and NewsData.io.

IX. CONCLUSION

NewsSync-AI demonstrates that a low-cost, zero-shot LLM can act as a real-time reranking *and* explainable-AI engine for personalised news, replacing resource-intensive offline training with on-the-fly profile synthesis. The architecture is layered, the API surface is minimal, the persistence schema is friendly to fast iteration, and the prompt-engineering protocol is simple enough to be reproduced by a small team. We hope the architecture specification and the two parallel iterations (Ollama for privacy, Gemini for accuracy) serve as a usable template for future news-intelligence experiments.

REFERENCES

- [1] C. Feng, M. Khan, A. U. Rahman and A. Ahmad, "News recommendation systems: Accomplishments, challenges and future directions," *IEEE Access*, vol. 8, pp. 167083–167104, 2020.
- [2] C. Wu, F. Wu, Y. Huang and X. Xie, "Personalized news recommendation: Methods and challenges," *ACM Transactions on Information Systems*, vol. 41, no. 3, pp. 1–45, 2023.
- [3] S. Raza and C. Ding, "News recommender system: A review of recent progress, challenges, and opportunities," *Artificial Intelligence Review*, vol. 55, pp. 749–800, 2022.
- [4] N. E. G. Mmaduakonam *et al.*, "Hybrid artificial intelligence approaches for cold-start mitigation in personalized news recommendation: A survey," *IPS Journal of Physical Sciences*, 2026.
- [5] Y. Wang, H. Sun and S. Zhang, "LLM as explainable re-ranker for recommendation system," *arXiv preprint arXiv:2512.03439*, 2025.
- [6] T. Pandit, S. Mahendru and M. Raval, "The evolution of reranking models in information retrieval: From heuristic methods to large language models," *Natural Language Processing and Information Retrieval (Springer)*, 2025.
- [7] Y. Zhang, S. Qiao, Z. Zhang, T. H. Lin and C. Gao, "A survey of large language model empowered agents for recommendation and search: Towards next-generation information retrieval," *ACM Transactions on Information Systems*, 2025.
- [8] J. Chen, Z. Liu, X. Huang, C. Wu, Q. Liu, C. Ge and W. Ou, "When large language models meet personalization: Perspectives of challenges and opportunities," *World Wide Web*, vol. 27, 2024.
- [9] Google DeepMind, "Gemini 1.5 Flash model card," 2024. [Online]. Available: <https://aistudio.google.com/>
- [10] Meta AI, "Llama 3.2 open model license," 2024. [Online]. Available: <https://llama.meta.com/llama3/license/>
- [11] NewsData.io, "Real-time global news API," 2024. [Online]. Available: <https://newsdata.io/>
- [12] Express.js Foundation, "Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: <https://expressjs.com/>
- [13] MongoDB Inc., "MongoDB documentation," 2024. [Online]. Available: <https://www.mongodb.com/docs/>
- [14] R. Kumar and A. K. R., "NewsSync-AI repository," GitHub, 2026. [Online]. Available: <https://github.com/Arakes101/NewsLens-AI>